

NLP 053 - CSE UOI 2025

Link prediction

École Polytechnique

Team Members:

Marios Ellinidis 4926

Nikolaos Konstantinidis 5155

1 Preprocessing

1.1 Graph Preprocessing

Στο πρόβλημα πρόβλεψης παραπομπών (citation prediction), το αρχείο `edgelist.txt` περιέχει τα θετικά δείγματα, δηλαδή ζεύγη κόμβων (*source, target*) όπου **υπάρχει παραπομπή**, ως πρώτο βήμα μπήκαμε στην διαδικασία παραγωγής αρνητικών ακμών .

Γιατί χρειάζονται αρνητικά ζεύγη:

- Στην εποπτευόμενη μάθηση (supervised learning) για δυαδική ταξινόμηση, το μοντέλο πρέπει να μάθει να διαχωρίζει δύο κλάσεις:
 - **Θετικά δείγματα:** ζεύγη με παραπομπή
 - **Αρνητικά δείγματα:** ζεύγη χωρίς παραπομπή
- Χρειάζεται να δημιουργήσουμε αρνητικά δείγματα ώστε το μοντέλο να μάθει πώς είναι η έλλειψη παραπομπής.
- Για την εξαγωγή χαρακτηριστικών (feature engineering), χρειάζονται παραδείγματα και από τις δύο κλάσεις για να εκπαιδευτεί σωστά ο Classifier.

Πώς δημιουργούμε αρνητικά ζεύγη:

- Επιλέγουμε τυχαία ζεύγη κόμβων (u, v) που **δεν έχουν ακμή** (δηλαδή δεν υπάρχει παραπομπή μεταξύ τους).
- Αποφεύγουμε ζεύγη που είναι στην πραγματικότητα θετικά, ώστε να μην έχουμε ψευδώς αρνητικά δείγματα, και τα αποθηκεύουμε στο αρχείο `negative_edgelist.txt`. Σε μετέπειτα κελιά στο Jupyter Notebook έχουμε εξελίξει το αρχείο με τις αρνητικές ακμές έτσι ώστε να μην εμπεριέχονται ούτε στο `test.txt`, καθώς μια ακμή προς ταξινόμηση θα μπορούσε λανθασμένα να αποτελεί μέρος των αρνητικών ακμών ενώ αποτελεί ακμή προς εξακρίβωση. Το αρχείο με τις νέες αρνητικές ακμές ονομάζεται `negative_edgelist2.txt`.

- Ο αριθμός των αρνητικών ζευγών είναι ίσος με των θετικών. Ισορροπώντας τον αριθμό των θετικών και αρνητικών δειγμάτων, το μοντέλο εκπαιδεύεται πιο δίκαια σε κάθε κλάση και αποφεύγουμε το bias προς την πλειοψηφούσα κλάση.

Συνοψίζοντας:

Για να εκπαιδεύσουμε ένα μοντέλο πρόβλεψης παραπομπών χρειαζόμαστε:

$$\text{Training set} = \underbrace{\{(u, v) \mid \text{υπάρχει παραπομπή}\}}_{\text{θετικά δείγματα}} \cup \underbrace{\{(x, y) \mid \text{δεν υπάρχει παραπομπή}\}}_{\text{αρνητικά δείγματα}}$$

και χαρακτηριστικά που εξάγονται για όλα τα ζεύγη, ώστε το μοντέλο να μάθει να διαχωρίζει πότε υπάρχει και πότε δεν υπάρχει παραπομπή.

```
import numpy as np

all_nodes = list(G.nodes)
negative_edges = set()

while len(negative_edges) < len(positive_edges):
    u, v = np.random.choice(all_nodes, 2, replace=False)
    if (u, v) not in positive_edges and (u, v) not in negative_edges
    and (u, v) not in test_edges:
        negative_edges.add((u, v))
        if len(negative_edges) % 100_000 == 0:
            print(f"{len(negative_edges)} negative edges generated so
                far...")

print(f"Positive examples: {len(positive_edges)}, Negative examples: {
    len(negative_edges)}")

with open("negative_edgelist.txt", "w") as f:
    for u, v in negative_edges:
        f.write(f"{u},{v}\n")

print("Negative edges saved to 'negative_edgelist.txt'")
```

Listing 1: Δημιουργία αρνητικών ζευγών για το πρόβλημα πρόβλεψης παραπομπών

```

import numpy as np

# Step 1: Load negative edges
negative_edges = set()
with open("negative_edgelist2.txt", "r") as f:
    for line in f:
        u, v = map(int, line.strip().split(','))
        negative_edges.add((u, v))

# Step 2: Load test edges
test_edges = []
with open("test.txt", "r") as f:
    for line in f:
        a, b = map(int, line.strip().split(","))
        test_edges.append((a, b))

# Step 3: Remove test edges from negative edges
for edge in test_edges:
    if edge in negative_edges:
        negative_edges.remove(edge)
print(f"Updated negative_edges size: {len(negative_edges)}")
# Step 4: Load positive edges and all nodes from edgelist.txt

```

Listing 2: Αναδημιουργία αρνητικών ακμών έτσι ώστε να μην υπάρχουν στο test.txt αρχείο

```

edges = pd.read_csv("edgelist.txt", header=None, names=["source", "target"])
positive_edges = list(edges.itertuples(index=False, name=None))
G = nx.DiGraph()
G.add_edges_from(edges.itertuples(index=False, name=None))

```

Listing 3: Φόρτωση θετικών ακμών και κατασκευή κατευθυνόμενου γράφου

1.2 Text Preprocessing

Για τα κείμενα των papers μας δίνεται το αρχείο abstracts.txt. Το πρόβλημα με αυτό το αρχείο ήταν ότι κάποια abstracts ήταν κενά, γεγονός που δυσχέραινε τη χρήση τους για εξαγωγή χαρακτηριστικών.

Για να το αντιμετωπίσουμε αυτό, μετατρέψαμε όλα τα abstracts σε *word2vec embeddings* χρησιμοποιώντας το προεκπαιδευμένο μοντέλο GoogleNews-vectors-negative300.bin:

```

from gensim.models import KeyedVectors

w2v = KeyedVectors.load_word2vec_format("GoogleNews-vectors-negative300.bin", binary=True)

```

Listing 4: Φόρτωση του προεκπαιδευμένου Word2Vec μοντέλου

Παρατηρήσαμε ότι τα embeddings των κενών abstracts ήταν γεμισμένα με μηδενικά. Για να αντιμετωπίσουμε το πρόβλημα αυτό, υλοποιήσαμε μία μέθοδο που εντοπίζει αυτά τα κενά embeddings και τα αντικαθιστά με embeddings βασισμένα στα abstracts που έχουν γραφτεί από κοινούς authors.

Η παρακάτω συνάρτηση υπολογίζει το embedding ενός paper με βάση τα embeddings των άλλων papers που έχουν τουλάχιστον έναν κοινό συγγραφέα:

```
def get_author_based_embedding(pid, authors, doc_embeddings,
                               embedding_dim=300):
    author_set = authors.get(pid, set())
    author_docs = [
        doc_id for doc_id, auth_set in authors.items()
        if doc_id != pid and auth_set & author_set
    ]
    valid_embeddings = [
        doc_embeddings[doc_id]
        for doc_id in author_docs
        if np.linalg.norm(doc_embeddings[doc_id]) > 0
    ]
    return np.mean(valid_embeddings, axis=0) if valid_embeddings else
        np.zeros(embedding_dim)
```

Listing 5: Συνάρτηση για υπολογισμό author-based embedding

Με αυτόν τον τρόπο, αν το abstract ενός paper είναι κενό, το embedding του αντικαθίσταται από το μέσο όρο των embeddings των abstracts άλλων papers με κοινό συγγραφέα, προσφέροντας μια πιο αντιπροσωπευτική αναπαράσταση.

Χαρακτηριστικά (Features) που χρησιμοποιήθηκαν:

- Ο αριθμός των κοινών γειτόνων μεταξύ των κόμβων a και b στο μη κατευθυνόμενο γράφο G :

$$\text{Common Neighbors} = |\Gamma(a) \cap \Gamma(b)|$$

- Η συνημίτονα ομοιότητα (cosine similarity) μεταξύ των word2vec embeddings των abstracts των κόμβων a και b .
- Ο αριθμός των κοινών συγγραφέων (authors) μεταξύ των κόμβων a και b :

$$|\text{authors}(a) \cap \text{authors}(b)|$$

Αποτέλεσμα: Με τη χρήση των παραπάνω χαρακτηριστικών και της τεχνικής αντικατάστασης κενών embeddings με author-based embeddings, ένα απλό μοντέλο LogisticRegression πέτυχε μέσο όρο *Log Loss* σε 5-fold cross-validation:

$$\text{Average Log Loss over 5 folds} = 0.2642$$

Αυτό το αποτέλεσμα υποδηλώνει βελτιωμένη απόδοση και πιο αξιόπιστη πρόβλεψη παραπομπών απο το να αφαιρέσουμε εντελώς τα abstracts αυτά ή απο το να τα αφήσουμε τα embeddings τους γεμισμένα με μηδενικά .

Δεύτερη τεχνική: Μέση τιμή embeddings γειτόνων (Fallback)

Για την αντιμετώπιση των μηδενικών embeddings, εφαρμόστηκε η μέθοδος αντικατάστασης των embeddings αυτών με τον μέσο όρο των embeddings των γειτονικών κόμβων στον γράφο. Συγκεκριμένα, ορίζουμε τη συνάρτηση:

```
def get_average_neighbor_embedding(node, graph, doc_embeddings,
    embedding_dim=300):
    neighbors = list(graph.neighbors(node))
    neighbor_embeds = [
        doc_embeddings[n]
        for n in neighbors if n in doc_embeddings and np.linalg.norm(
            doc_embeddings[n]) > 0
    ]
    if neighbor_embeds:
        return np.mean(neighbor_embeds, axis=0)
    else:
        return np.zeros(embedding_dim)
```

Το αποτέλεσμα αυτής της τεχνικής, αξιολογημένο με 5-fold cross-validation, ήταν:

Average Log Loss over 5 folds = 0.2553

Η βελτίωση αυτή σε σχέση με την προηγούμενη τεχνική δείχνει ότι η χρήση embeddings γειτόνων για την αντικατάσταση των κενών abstracts βοηθάει στην καλύτερη εκμάθηση του μοντέλου.

Τρίτη τεχνική: Υβριδικό embedding με βάση συγγραφείς και γείτονες

Για να αξιοποιηθεί η πληροφορία τόσο από τους συγγραφείς όσο και από τους γείτονες στον γράφο, δημιουργήθηκε ένα υβριδικό embedding μέσω σταθμισμένου μέσου όρου. Ο συνδυασμός των δύο τεχνικών γίνεται με τη χρήση δύο βαρών: w_{neigh} για τα embeddings των γειτόνων και w_{auth} για τα embeddings από κοινούς συγγραφείς. Η υλοποίηση είναι η εξής:

```
# Author-based embedding
def get_author_based_embedding(pid, authors, doc_embeddings,
    embedding_dim=300):
    author_set = authors.get(pid, set())
    author_docs = [
        doc_id for doc_id, auth_set in authors.items()
        if doc_id != pid and auth_set & author_set
    ]
    valid_embeddings = [
        doc_embeddings[doc_id]
        for doc_id in author_docs
        if np.linalg.norm(doc_embeddings[doc_id]) > 0
    ]
    return np.mean(valid_embeddings, axis=0) if valid_embeddings else
        np.zeros(embedding_dim)

# Neighbor-based embedding
def get_average_neighbor_embedding(node, graph, doc_embeddings,
    embedding_dim=300):
    neighbors = list(graph.neighbors(node))
    neighbor_embeds = [
        doc_embeddings[n]
        for n in neighbors if n in doc_embeddings and np.linalg.norm(
            doc_embeddings[n]) > 0
    ]
    return np.mean(neighbor_embeds, axis=0) if neighbor_embeds else np.
        zeros(embedding_dim)

# Hybrid embedding: weighted average
def get_hybrid_embedding(pid, graph, authors, doc_embeddings,
    embedding_dim=300, w_neigh=0.6, w_auth=0.4):
    neighbor_emb = get_average_neighbor_embedding(pid, graph,
        doc_embeddings, embedding_dim)
    author_emb = get_author_based_embedding(pid, authors,
        doc_embeddings, embedding_dim)
    if np.linalg.norm(neighbor_emb) == 0 and np.linalg.norm(author_emb)
        == 0:
        return np.zeros(embedding_dim)
    return w_neigh * neighbor_emb + w_auth * author_emb
```

Για την υλοποίηση χρησιμοποιήθηκαν τιμές $w_{\text{neigh}} = 0.6$ και $w_{\text{auth}} = 0.4$, ώστε να δοθεί μεγαλύτερη βαρύτητα στην πληροφορία των γειτόνων. Το τελικό αποτέλεσμα αυτής της προσέγγισης, με 5-fold cross-validation, ήταν:

$$\text{Average Log Loss over 5 folds} = 0.2571$$

Παρατηρείται ότι η υβριδική προσέγγιση δεν ξεπέρασε τη μέθοδο των αποκλειστικά γειτονικών embeddings, αλλά ίσως αν αλλάζαμε την βαρύτητα των 2 τεχνικών να πετυχαίναμε ένα καλύτερο αποτέλεσμα

Τέταρτη τεχνική: Υβριδική fallback προσέγγιση για κενά embeddings

Κατά την ανάλυση των δεδομένων παρατηρήθηκε ότι τόσο η τεχνική βασισμένη στους γείτονες όσο και εκείνη με βάση τους κοινούς συγγραφείς δεν επαρκούν για την πλήρη κάλυψη των περιπτώσεων όπου τα αρχικά embeddings των εγγράφων (από τα abstract) είναι μηδενικά. Πράγματι, ορισμένα άρθρα είτε δεν διαθέτουν γείτονες στον γράφο αναφορών είτε δεν έχουν κοινούς συγγραφείς με άλλα έγγραφα.

Για την αντιμετώπιση αυτού του ζητήματος, υλοποιήθηκε μια **υβριδική fallback στρατηγική**, η οποία ακολουθεί την εξής ιεραρχία:

1. Πρώτα ελέγχεται αν το έγγραφο έχει μη μηδενικό embedding από το *abstract*.
2. Αν όχι, εφαρμόζεται η τεχνική των **average neighbor embeddings**.
3. Αν παραμένει μηδενικό το embedding, εφαρμόζεται η τεχνική των **author-based embeddings**.
4. Αν καμία από τις παραπάνω τεχνικές δεν αποδώσει έγκυρο embedding, επιστρέφεται μηδενικό διάνυσμα.

Ο αντίστοιχος κώδικας της προσέγγισης είναι:

```
embedding_stats = {
    "abstract": 0,
    "neighbor_fallback": 0,
    "author_fallback": 0,
    "zero_vector": 0
}

def get_embedding(text):
    words = text.split()
    vectors = [w2v[w] for w in words if w in w2v]
    return np.mean(vectors, axis=0) if vectors else np.zeros(w2v.
        vector_size)

# Create document embeddings from abstracts
doc_embeddings = {}
for pid, text in tqdm(abstracts.items(), desc="Embedding documents"):
    doc_embeddings[pid] = get_embedding(text)

# Hybrid fallback embedding
def get_hybrid_embedding(pid, graph, authors, doc_embeddings, dim=300):
    emb = doc_embeddings.get(pid)
    embedding_stats["abstract"] += 1
    if emb is not None and np.linalg.norm(emb) > 0:
        return emb

    emb = get_average_neighbor_embedding(pid, graph, doc_embeddings,
        dim)
    if np.linalg.norm(emb) > 0:
        embedding_stats["neighbor_fallback"] += 1
        return emb

    emb = get_author_based_embedding(pid, authors, doc_embeddings)
    if np.linalg.norm(emb) > 0:
        embedding_stats["author_fallback"] += 1
        return emb

    embedding_stats["zero_vector"] += 1
    return emb
```

Η παραπάνω τεχνική επέτρεψε την κάλυψη μεγαλύτερου ποσοστού των εγγράφων και τη μείωση των μηδενικών embeddings. Το τελικό αποτέλεσμα με αυτήν τη στρατηγική, με χρήση 5-fold cross-validation, ήταν:

Average Log Loss over 5 folds = **0.2538**

Σημείωση: Δοκιμάστηκε επίσης μια εναλλακτική εκδοχή της υβριδικής στρατηγικής όπου η ιεραρχία των fallback embeddings αντιστράφηκε, δηλαδή δοκιμάστηκε πρώτα η τεχνική των *author-based embeddings* και στη συνέχεια εκείνη των *neighbor-based embeddings*. Ωστόσο, η προσέγγιση αυτή απέδωσε χειρότερα αποτελέσματα, με:

Average Log Loss over 5 folds = **0.2597**

γεγονός που επιβεβαιώνει την αποτελεσματικότητα της αρχικής σειράς προτεραιότητας (πρώτα γείτονες και μετά συγγραφείς).

Πολυεπίπεδη Στρατηγική Fallback (Multi-fallback Embedding Strategy)

Παρά τη χρήση της υβριδικής στρατηγικής με fallback βασισμένο αρχικά στους γείτονες και στη συνέχεια στους κοινούς συγγραφείς, διαπιστώθηκε ότι εξακολουθούν να υπάρχουν περιπτώσεις όπου ο τελικός embedding παραμένει μηδενικό. Συγκεκριμένα, παρατηρήθηκε ότι για αρκετές περιπτώσεις δεν υπήρχαν διαθέσιμα ούτε κοινά abstracts, ούτε κοινοί authors, ούτε γείτονες στο citation γράφο.

Για την αντιμετώπιση αυτών των περιπτώσεων, εφαρμόστηκε μια βελτιωμένη **πολυεπίπεδη στρατηγική fallback** με την εξής ιεραρχία:

1. **Άμεσο embedding από το abstract** (μέσος όρος των Word2Vec embeddings).
2. **1-hop neighbor-based fallback:** χρήση των embeddings των απευθείας γειτόνων στο citation γράφο.
3. **2-hop neighbor-based fallback:** χρήση των embeddings των γειτόνων των γειτόνων (π.χ. “ξαδέρφια”).
4. **Author-based fallback:** χρήση των embeddings άλλων papers με κοινούς συγγραφείς.
5. **Global mean fallback:** αν όλες οι παραπάνω τεχνικές αποτύχουν, χρησιμοποιείται ο μέσος όρος όλων των μη μηδενικών embeddings.

Η παρακάτω Python υλοποίηση χρησιμοποιήθηκε για την κατασκευή των embedding:

```
# Precompute global mean
valid_embeds = [v for v in doc_embeddings.values() if np.linalg.norm(v) > 0]
global_mean = np.mean(valid_embeds, axis=0) if valid_embeds else np.zeros(w2v.
    vector_size)

# Author-based embedding
def get_author_based_embedding(pid, authors, doc_embeddings, embedding_dim
    =300):
    author_set = authors.get(pid, set())
    author_docs = [
        doc_id for doc_id, auth_set in authors.items()
        if doc_id != pid and auth_set & author_set
    ]
    valid_embeddings = [
        doc_embeddings[doc_id]
        for doc_id in author_docs
        if np.linalg.norm(doc_embeddings[doc_id]) > 0
    ]
    return np.mean(valid_embeddings, axis=0) if valid_embeddings else np.zeros(
        embedding_dim)

# 1-hop neighbor-based embedding
def get_average_neighbor_embedding(node, graph, doc_embeddings, embedding_dim
    =300):
    neighbors = list(graph.neighbors(node))
    neighbor_embeds = [
        doc_embeddings[n]
        for n in neighbors if n in doc_embeddings and np.linalg.norm(
            doc_embeddings[n]) > 0
    ]
    return np.mean(neighbor_embeds, axis=0) if neighbor_embeds else np.zeros(
        embedding_dim)

# 2-hop neighbor-based embedding
def get_2hop_neighbor_embedding(node, graph, doc_embeddings, embedding_dim
    =300):
    two_hop_neighbors = set()
    for neighbor in graph.neighbors(node):
        two_hop_neighbors.update(graph.neighbors(neighbor))
    two_hop_neighbors.discard(node)
    valid_embeddings = [
        doc_embeddings[n]
        for n in two_hop_neighbors if n in doc_embeddings and np.linalg.norm(
            doc_embeddings[n]) > 0
    ]
    return np.mean(valid_embeddings, axis=0) if valid_embeddings else np.zeros(
        embedding_dim)

# Hybrid embedding with full fallback chain
def get_hybrid_embedding(pid, graph, authors, doc_embeddings, dim=300):
    emb = doc_embeddings.get(pid)
    if emb is not None and np.linalg.norm(emb) > 0:
```

```

        embedding_stats["abstract"] += 1
        return emb

    emb = get_average_neighbor_embedding(pid, graph, doc_embeddings, dim)
    if np.linalg.norm(emb) > 0:
        embedding_stats["neighbor_fallback"] += 1
        return emb

    emb = get_2hop_neighbor_embedding(pid, graph, doc_embeddings, dim)
    if np.linalg.norm(emb) > 0:
        embedding_stats["2hop_fallback"] += 1
        return emb

    emb = get_author_based_embedding(pid, authors, doc_embeddings, dim)
    if np.linalg.norm(emb) > 0:
        embedding_stats["author_fallback"] += 1
        return emb

    if np.linalg.norm(global_mean) > 0:
        embedding_stats["global_mean_fallback"] += 1
        return global_mean

    embedding_stats["zero_vector"] += 1
    return np.zeros(dim)

```

Listing 6: Hybrid Embedding Strategy with Multi-level Fallback

Η τελική κατανομή της πηγής των embeddings μετά την εφαρμογή της παραπάνω στρατηγικής ήταν η εξής:

```

=== Embedding Source Statistics ===
Abstract                : 4408323
Neighbor fallback       : 166998
2hop fallback           : 5817
Author fallback         : 54
Global mean fallback    : 12
Zero vector             : 0

```

Η στρατηγική αυτή οδήγησε σε περαιτέρω βελτίωση της απόδοσης:

Average Log Loss over 5 folds = **0.2535**

δηλαδή μια μικρή αλλά μετρήσιμη βελτίωση σε σχέση με την προηγούμενη υλοποίηση χωρίς το 2-hop και το global mean fallback.

Σημείωση: Όλες οι παραπάνω μετρήσεις μέχρι στιγμής έγιναν με Logistic Regression ως classifier

2 Feature Engineering

2.1 Graph-Based Features [13]

Jaccard Similarity μεταξύ Γειτόνων

Μια από τις πρώτες γραφοβασισμένες τεχνικές που εξετάστηκαν ήταν η **Jaccard similarity** μεταξύ των συνόλων γειτόνων δύο κόμβων. Η ιδέα βασίζεται στο ότι δύο κόμβοι (papers) που μοιράζονται παρόμοιο σύνολο γειτόνων (δηλαδή κοινές αναφορές ή κοινά citations) είναι πιθανότερο να έχουν άμεση σχέση.

Ο υπολογισμός της Jaccard similarity βασίζεται στον τύπο:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

όπου A και B είναι τα σύνολα των γειτόνων των κόμβων a και b , αντίστοιχα. Η αντίστοιχη Python υλοποίηση:

```
def jaccard_similarity(set_a, set_b):  
    intersection = len(set_a & set_b)  
    union = len(set_a | set_b)  
    return intersection / union if union != 0 else 0.0
```

Listing 7: Jaccard Similarity μεταξύ γειτόνων

Η Jaccard similarity ενσωματώθηκε στο τελικό feature vector ως εξής:

```
X.append([  
    jaccard_similarity(a_neighbors, b_neighbors),  
    cosine_similarity([a_emb], [b_emb])[0][0],  
    len(authors.get(a, set()) & authors.get(b, set())),  
])
```

Παρότι η συγκεκριμένη τεχνική είναι απλή και ευρέως χρησιμοποιούμενη σε προβλήματα link prediction, στην περίπτωση μας **δεν οδήγησε σε βελτίωση του τελικού σκορ**. Πιθανή εξήγηση είναι ότι η πληροφορία που μεταφέρει καλύπτεται ήδη από τα υπόλοιπα features, όπως το **cosine similarity των embeddings** και οι **κοινοί συγγραφείς**.

Adamic-Adar Index

Μια πιο informative παραλλαγή της τεχνικής “common neighbors” είναι ο **Adamic-Adar index**, ο οποίος λαμβάνει υπόψη όχι μόνο την ύπαρξη κοινών γειτόνων μεταξύ δύο κόμβων, αλλά και τη σπανιότητά τους. Ορίζεται ως:

$$AA(a, b) = \sum_{z \in \Gamma(a) \cap \Gamma(b)} \frac{1}{\log |\Gamma(z)|}$$

όπου $\Gamma(x)$ δηλώνει το σύνολο γειτόνων του κόμβου x , και z είναι οι κοινοί γείτονες των a και b . Η λογική πίσω από τον δείκτη αυτό είναι πως ένας κοινός γείτονας με μικρό βαθμό (δηλαδή, που συνδέεται με λίγους κόμβους) φέρει μεγαλύτερη πληροφορία για πιθανή σύνδεση, σε σύγκριση με έναν κοινό γείτονα με μεγάλο βαθμό.

Η υλοποίηση βασίστηκε στη συνάρτηση `adamic_adar_index` του `NetworkX`:

```
aa_score = list(nx.adamic_adar_index(undirected_G, [(a, b)]))[0][2]
```

Listing 8: Adamic-Adar feature μεταξύ κόμβων

Το χαρακτηριστικό αυτό ενσωματώθηκε στο feature vector αντικαθιστώντας το παλαιότερο “common neighbors” feature, καθώς παρατηρήθηκε ότι:

- Το **Adamic-Adar είναι αυστηρή βελτίωση του common neighbors**, εισάγοντας βάρη στους κοινούς γείτονες βάσει του βαθμού τους.
- Η χρήση και των δύο χαρακτηριστικών μαζί οδηγούσε σε πλεονασμό πληροφορίας (feature redundancy), χωρίς μετρήσιμη βελτίωση.
- Η αφαίρεση του “common neighbors” και η διατήρηση μόνο του Adamic-Adar είχε ως αποτέλεσμα **την βελτίωση στην απόδοση του μοντέλου**.

Συγκεκριμένα, το μέσο log loss σε 5-fold cross validation μειώθηκε από **0.2535 σε 0.2504**, επιβεβαιώνοντας την θετική συνεισφορά του Adamic-Adar στο τελικό feature set.

Preferential Attachment

Μια ακόμη σημαντική τοπολογική μέθοδος που ενσωματώθηκε στο σύνολο χαρακτηριστικών είναι η **Preferential Attachment (PA)**. Η τεχνική αυτή βασίζεται στη θεωρία των δικτύων Barabási–Albert και υποθέτει ότι κόμβοι με υψηλό βαθμό (δηλαδή μεγάλο πλήθος ακμών) έχουν μεγαλύτερη πιθανότητα να δημιουργήσουν νέες συνδέσεις.

Ο δείκτης Preferential Attachment μεταξύ δύο κόμβων a και b ορίζεται ως το γινόμενο των βαθμών τους:

$$PA(a, b) = |\Gamma(a)| \cdot |\Gamma(b)|$$

Η υλοποίηση πραγματοποιήθηκε μέσω της συνάρτησης `preferential_attachment` από το NetworkX:

```
def compute_preferential_attachment(graph, u, v):  
    try:  
        return list(preferential_attachment(graph, [(u, v)]))[0][2]  
    except:  
        return 0.0
```

Listing 9: Υπολογισμός Preferential Attachment

Το χαρακτηριστικό αυτό προστέθηκε στο υπάρχον σύνολο γραφο-βασισμένων χαρακτηριστικών, ενισχύοντας την πληροφορία που φέρουν οι τοπολογικές σχέσεις μεταξύ κόμβων.

Αποτέλεσμα: Η προσθήκη του δείκτη Preferential Attachment οδήγησε σε **σημαντική βελτίωση της απόδοσης του μοντέλου**, με το μέσο **log loss να μειώνεται από 0.2504 σε 0.2338** σε 5-fold cross validation. Το αποτέλεσμα αυτό καταδεικνύει ότι η πληροφορία που ενσωματώνει η μέθοδος σχετικά με την “προτίμηση σύνδεσης” υψηλόβαθμων κόμβων είναι χρήσιμη για την πρόβλεψη ακμών στο γράφο αναφορών.

Σημείωση: δοκιμάστηκε να ξαναπροσθέσουμε το jaccard similarity στα νέα δεδομένα αλλά και πάλι η παρουσία του κράτησε το σκορ σταθερά στο 0.2338 γεγονός που το καθιστά πρακτικά ασήμαντο για να το χρησιμοποιήσουμε ,αυξάνει το υπολογιστικό κόστος χωρίς κάποιο όφελος οπότε αφαιρέθηκε

Centrality-Based Features[6]

Μία σημαντική κατηγορία χαρακτηριστικών που δοκιμάσαμε βασίζεται στους *κεντρικότητες* των κόμβων στον γράφο αναφορών. Αυτά τα χαρακτηριστικά αποτυπώνουν τη σχετική σημασία ή επιρροή ενός κόμβου με βάση τη θέση του στο δίκτυο. Αρχικά, υπολογίσαμε και προσθέσαμε τα εξής:

- **Closeness Centrality:** Μετρά πόσο "κοντά" είναι ένας κόμβος σε όλους τους υπόλοιπους στο δίκτυο, ως το αντίστροφο του μέσου μήκους των συντομότερων μονοπατιών από τον κόμβο προς όλους τους άλλους. Ο μαθηματικός ορισμός για έναν κόμβο v είναι:

$$C_{closeness}(v) = \frac{1}{\sum_{u \neq v} d(u, v)}$$

όπου $d(u, v)$ είναι η απόσταση μεταξύ των κόμβων u και v . Αν και ο υπολογισμός του ήταν ιδιαίτερα χρονοβόρος για τον γράφο μας των 138,499 κόμβων και 1,091,955 ακμών, τελικά ενίσχυσε τη συνολική απόδοση του μοντέλου. Ο συνδυασμός του με τα υπάρχοντα χαρακτηριστικά (Adamic-Adar, Preferential Attachment, συνεμφάνιση συγγραφέων και συνεφαπτόμενα embeddings) μείωσε το **Average Log Loss over 5 folds** από 0.2338 σε 0.2271.

- **Betweenness Centrality:** Υπολογίστηκε επίσης, μετρώντας για κάθε κόμβο το πλήθος των shortest paths που περνούν από αυτόν. Ωστόσο, δεν επέφερε αισθητή βελτίωση στην απόδοση και δεν συμπεριλήφθηκε τελικά.
- **Degree Centrality:** Ο βαθμός κεντρικότητας ορίζεται ως:

$$C_{degree}(v) = \frac{\deg(v)}{n - 1}$$

δηλαδή το ποσοστό κόμβων με τους οποίους συνδέεται ο κόμβος v , όπου $\deg(v)$ είναι ο βαθμός (αριθμός γειτόνων) του κόμβου v και n ο συνολικός αριθμός κόμβων, δεν χρησιμοποιήθηκε εν τέλει λόγω υπολογιστικού κόστους

- **Άλλες μέθοδοι:** Δοκιμάστηκαν επίσης τα:

- Eigenvector Centrality
- PageRank
- Katz Centrality
- HITS / Birt Score

Οι περισσότερες από αυτές ήταν υπολογιστικά πολύ απαιτητικές για τον συγκεκριμένο γράφο, με αποτέλεσμα είτε να εγκαταλειφθούν λόγω μεγάλου χρόνου υπολογισμού, είτε επειδή δεν βελτιώναν το τελικό αποτέλεσμα.

Η τελική επιλογή των χαρακτηριστικών βασίστηκε τόσο στην προβλεπτική ισχύ τους όσο και στην αποδοτικότητα υπολογισμού τους. Έτσι, το σύνολο χαρακτηριστικών που οδήγησε στην καλύτερη επίδοση περιελάμβανε:

- Closeness Centrality των κόμβων a και b
- Adamic-Adar μεταξύ a και b
- Preferential Attachment μεταξύ a και b
- Συνεμφάνιση συγγραφέων
- Συνεφαπτόμενα embeddings (cosine similarity)

Embeddings Transformation and Clustering Coefficient

Στο επόμενο στάδιο, επικεντρωθήκαμε στη βελτιστοποίηση της αναπαράστασης των κειμενικών χαρακτηριστικών. Τα αρχικά embeddings που χρησιμοποιήθηκαν ήταν Word2Vec vectors 300 διαστάσεων (GoogleNews-vectors-negative300.bin)[15]. Αν και δεν επιλέξαμε να μειώσουμε τη διάστασή τους, εφαρμόσαμε **Principal Component Analysis (PCA)** [16] για να αναπροσανατολίσουμε τον χώρο των χαρακτηριστικών. Ο κώδικας που χρησιμοποιήθηκε ήταν:

```
pca = PCA(n_components=300)
reduced_embeddings = pca.fit_transform(embeddings_matrix)
```

Το **PCA** είναι μία τεχνική μείωσης διαστάσεων που βασίζεται στη γραμμική άλγεβρα και έχει ως στόχο να εντοπίσει τις κατευθύνσεις (κύριες συνιστώσες) στις οποίες τα δεδομένα έχουν τη μεγαλύτερη διακύμανση. Ακόμη και αν δεν μειώνεται η διάσταση, το *reorientation* των δεδομένων μπορεί να ενισχύσει τη δυνατότητα του μοντέλου να διαχωρίσει καλύτερα τις κατηγορίες, καθώς εξαλείφονται στατιστικές συσχετίσεις μεταξύ διαστάσεων.

Μετά την εφαρμογή του PCA στα Word2Vec embeddings, παρατηρήθηκε σημαντική βελτίωση με **Average Log Loss over 5 folds: 0.1810**.

Επιπλέον, προστέθηκε στα χαρακτηριστικά και ο **Clustering Coefficient**[19] για κάθε κόμβο. Ο τοπικός συντελεστής συσσωμάτωσης (*clustering coefficient*) μετρά τον βαθμό στον οποίο οι γείτονες ενός κόμβου είναι μεταξύ τους συνδεδεμένοι, ορίζεται ως:

$$C(v) = \frac{2T(v)}{k_v(k_v - 1)}$$

όπου $T(v)$ είναι ο αριθμός τριγώνων που περιλαμβάνουν τον κόμβο v και k_v είναι ο βαθμός του κόμβου.

Ο συνδυασμός των ανανεωμένων embeddings μέσω PCA και του clustering coefficient με τα υπόλοιπα χαρακτηριστικά:

- Closeness Centrality των κόμβων a, b
- Adamic-Adar δείκτης μεταξύ a και b
- Preferential Attachment
- Cosine Similarity των κειμενικών embeddings
- Συγγραφική επικάλυψη
- Clustering Coefficient των κόμβων a, b

οδήγησε σε σημαντική μείωση του σφάλματος, με τελικό **Average Negative Log-Likelihood (Log Loss): 0.1651**.

2.2 Textual Features

Για τη μοντελοποίηση των κειμενικών χαρακτηριστικών των ερευνητικών άρθρων, πειραματιστήκαμε αρχικά με διαφορετικούς τύπους embeddings. Πέρα από τις παραδοσιακές τεχνικές όπως τα TF-IDF και Word2Vec, δοκιμάστηκαν και πιο σύγχρονες μέθοδοι βασισμένες σε μοντέλα μετασχηματιστών (Transformers).

SciBERT Embeddings.

Μία από τις πρώτες προσεγγίσεις που δοκιμάστηκαν ήταν η χρήση των προεκπαιδευμένων **SciBERT** embeddings, τα οποία είναι ειδικά σχεδιασμένα για την αναπαράσταση επιστημονικών κειμένων [1]. Το SciBERT είναι παραλλαγή του BERT που έχει εκπαιδευτεί σε επιστημονικά άρθρα, και παρέχει πλούσια αναπαράσταση περιεχομένου βάσει σημασιολογίας.

Η μέθοδος εξαγωγής χαρακτηριστικών ακολουθεί την τυπική προσέγγιση χρήσης του [CLS] token ως συνολική αναπαράσταση της πρότασης (sentence-level embedding). Παράδειγμα κώδικα:

```
inputs = tokenizer(text, return_tensors='pt', truncation=True, padding=True, max_length=512)
outputs = model(**inputs)
embedding = outputs.last_hidden_state[:, 0, :].squeeze().numpy()
```

Ωστόσο, λόγω του μεγάλου αριθμού δεδομένων και του μεγάλου μεγέθους των SciBERT embeddings (768 διαστάσεις), παρουσιάστηκαν προβλήματα διαχείρισης μνήμης. Για την επίλυσή τους, τα embeddings υπολογίστηκαν τμηματικά (*chunked processing*) και αποθηκεύτηκαν σε ξεχωριστά αρχεία, ώστε να μπορούν να φορτωθούν και να χρησιμοποιηθούν σταδιακά.

Μείωση Διαστάσεων.

Μετά τη συλλογή όλων των SciBERT embeddings, εφαρμόστηκε **Principal Component Analysis (PCA)** με $n = 600$ κύριες συνιστώσες για μείωση διαστάσεων :

```
pca = PCA(n_components=600, random_state=42)
reduced_embeddings = pca.fit_transform(embedding_matrix)
```

Cosine Similarity μεταξύ abstracts. Τα μειωμένα embeddings χρησιμοποιήθηκαν για να υπολογιστεί η ομοιότητα μεταξύ δύο άρθρων μέσω *cosine similarity*, αντικαθιστώντας τα προηγούμενα Word2Vec embeddings GoogleNews-vectors-negative300:

```
cosine_similarity([a_sci], [b_sci])[0][0]
```

Αυτό το χαρακτηριστικό ενσωματώθηκε στο τελικό διανύσμα χαρακτηριστικών για κάθε ζεύγος (a, b) :

- Clustering Coefficient για τους κόμβους a, b
- Closeness Centrality για τους a, b
- Adamic-Adar δείκτης
- Preferential Attachment
- Cosine Similarity των SciBERT embeddings
- Επικάλυψη συγγραφέων

Αποτελέσματα. Παρά τις ελπίδες ότι το SciBERT θα προσέφερε σημαντική βελτίωση στην καταγραφή της σημασιολογικής πληροφορίας, τα τελικά αποτελέσματα ήταν ελαφρώς χειρότερα σε σχέση με τη χρήση των Word2Vec embeddings, με τελικό σκορ:

Average Negative Log-Likelihood over 5 folds: 0.1909

Αυτό μπορεί να αποδοθεί στην υπερβολικά υψηλή διάσταση, πιθανή υπερπροσαρμογή, ή στο ότι τα απλούστερα Word2Vec embeddings είχαν καλύτερη συσχέτιση με το task της πρόβλεψης παραπομπών, μια άλλη αιτία μπορεί να είναι ότι προκύπτει κάποιο σφάλμα κατά τον διαχωρισμό σε chunks. Δοκιμάστηκε να χρησιμοποιηθούν και τα 2 ειδών embeddings τα sciBERT embeddings μαζί με αυτά του GoogleNews-vectors-negative300, αλλά φάνηκε ότι δεν απέδιδαν καλύτερα αποτελέσματα από αυτά που είχαν τα embeddings GoogleNews-vectors-negative300 μόνα τους, οπότε αποφασίσαμε τελικά να μην χρησιμοποιήσουμε καθόλου τα sciBERT embeddings

Sentence Embeddings με Specter2.

Παρατηρώντας ότι τα **SciBERT** embeddings δεν αποτύπωναν τα αποτελέσματα που ελπίζαμε, στραφήκαμε σε μια πιο σύγχρονη και εξειδικευμένη προσέγγιση. Χρησιμοποιήσαμε το προεκπαιδευμένο μοντέλο `allenai/specter2_base` [4] https://huggingface.co/allenai/specter2_base το οποίο βασίζεται στην αρχιτεκτονική Transformers και έχει εκπαιδευτεί ειδικά για την παραγωγή αναπαραστάσεων επιστημονικών άρθρων με στόχο τη βελτιστοποίηση της ομοιότητας βάσει αναφορών (citation-based similarity).

Κάθε abstract μετατρέπεται σε tokens και περνάει μαζικά (batch-wise) στο μοντέλο, από το οποίο εξάγαμε το embedding του [CLS] token — μια συγκεντρωτική αναπαράσταση του κειμένου. Τα παραγόμενα embeddings αποθηκεύτηκαν για μετέπειτα χρήση.

Multifallback για Κενά Abstracts. Για τα abstracts που ήταν κενά, ακολουθήθηκε μια **Multifallback** στρατηγική παραγωγής αναπαραστάσεων όπως και με τα word2vec προκειμένου να διατηρηθεί η πληροφορία:

- χρήση embedding βάσει των authors (author-based)
- χρήση μέσου όρου embeddings των γειτονικών κόμβων στο citation graph (neighbor-based)
- ή άλλες εναλλακτικές όταν καμία από τις παραπάνω δεν ήταν διαθέσιμη.

Μείωση Διάστασης και Υπολογισμός Ομοιότητας. Η διάσταση των embeddings (αρχικά 768) μειώθηκε μέσω **PCA** σε $n_components = 250$ με στόχο τη μείωση θορύβου και τη βελτίωση της υπολογιστικής απόδοσης. Η σημασιολογική εγγύτητα μεταξύ δύο abstracts *a* και *b* μετρήθηκε τελικά με **cosine similarity** μεταξύ των αντίστοιχων Specter2 embeddings.

Αποτελέσματα. Η χρήση του Specter2 σε συνδυασμό με τη multifallback στρατηγική και τη μείωση διάστασης οδήγησε σε σημαντική αύξηση της επίδοσης, μειώνοντας τον **Negative Log-Likelihood** στους **0.1004** κατά μέσο όρο σε 5-fold cross-validation, βελτιώνοντας αισθητά τα αποτελέσματα σε σχέση με προηγούμενες προσεγγίσεις.

TF-IDF Similarity

Σε επόμενο βήμα , χρησιμοποιήσαμε τη μέθοδο **TF-IDF** (Term Frequency–Inverse Document Frequency)[14], η οποία δίνει βάρος στις λέξεις ανάλογα με τη σχετική τους συχνότητα στο έγγραφο και στο σύνολο εγγράφων. Έτσι, αποφεύγεται η υπερ-εκπροσώπηση κοινών λέξεων και τονίζονται όροι με υψηλή διακριτική ισχύ. Η υλοποίηση βασίστηκε στον παρακάτω κώδικα:

```
vectorizer = TfidfVectorizer(stop_words='english', max_features=10000)
tfidf_matrix = vectorizer.fit_transform(abstracts.values())
paper_id_to_tfidf = {i: tfidf_matrix[i] for i in range(len(abstracts))}
```

Για κάθε υποψήφιο ζεύγος εγγράφων (a, b), υπολογίσαμε την *TF-IDF similarity* ως **συσχέτιση συνημιτόνου (cosine similarity)** μεταξύ των διανυσμάτων τους:

```
tfidf_sim = cosine_similarity(a_tfidf, b_tfidf)[0][0]
```

Το χαρακτηριστικό αυτό ενσωματώθηκε στο τελικό διάνυσμα χαρακτηριστικών κάθε ζεύγους, όπως φαίνεται στον ακόλουθο κώδικα δημιουργίας feature vectors:

```
X.append([
    clustering_coeffs.get(a, 0.0),
    clustering_coeffs.get(b, 0.0),
    closeness centrality.get(a, 0.0),
    closeness centrality.get(b, 0.0),
    compute_adamic_adar(undirected_G, a, b),
    compute_preferential_attachment(undirected_G, a, b),
    cosine_similarity([a_w2v], [b_w2v])[0][0], # Word2Vec sim
    cosine_similarity([a_sci], [b_sci])[0][0], # Specter2 sim
    tfidf_sim,
    len(authors.get(a, set()) & authors.get(b, set()))
])
```

Η ενσωμάτωση του TF-IDF similarity συνέβαλε σε περαιτέρω βελτίωση του μοντέλου, οδηγώντας σε **μείωση του Average Negative Log-Likelihood σε 0.0956**, σε πενταπλή διασταυρούμενη επικύρωση.

Doc2Vec Embeddings.

Για την ενίσχυση της αναπαράστασης των περιλήψεων, αξιοποιήσαμε το **Doc2Vec**[12], μια μέθοδο που παράγει συνεκτικά διανύσματα (embeddings) για ολόκληρα έγγραφα. Σε αντίθεση με το Word2Vec, το οποίο παράγει διανύσματα για μεμονωμένες λέξεις, το Doc2Vec μαθαίνει αναπαραστάσεις που ενσωματώνουν τη συνολική σημασιολογική πληροφορία ενός κειμένου, ιδανικές για συγκρίσεις εγγράφων.

Αρχικά, κάθε περίληψη tokenizόταν και μετατρεπόταν σε αντικείμενο τύπου TaggedDocument, το οποίο απαιτείται από το Doc2Vec για να αναγνωρίζει το κείμενο και την αντίστοιχη “ετικέτα” (ID του άρθρου). Έπειτα, εκπαιδεύσαμε το μοντέλο με τις ακόλουθες ρυθμίσεις: μέγεθος διανύσματος 50, παράθυρο συμφοραζομένων 1, ελάχιστη συχνότητα εμφάνισης λέξεων 25 και χρήση της παραλλαγής Distributed Memory (dm=1).

```
from tqdm import tqdm
from gensim.models.doc2vec import TaggedDocument, Doc2Vec
from nltk.tokenize import word_tokenize

# Tokenization          abstracts   TaggedDocuments
tagged_docs = []
for paper_id, text in tqdm(abstracts.items(), desc="Tokenizing
abstracts"):
    tokens = word_tokenize(text.lower())
    tagged_docs.append(TaggedDocument(words=tokens, tags=[str(paper_id)
    ]))

# A                      Doc2Vec
doc2vec_model = Doc2Vec(
    vector_size=50,          #          embeddings
    window=1,               #
    min_count=25,           #          <25
    workers=4,              #          threads
    epochs=400,             #
    dm=1                    # Distributed Memory (   DBOW)
)

doc2vec_model.build_vocab(tagged_docs)
doc2vec_model.train(tagged_docs, total_examples=doc2vec_model.
    corpus_count, epochs=doc2vec_model.epochs)

# A
paper_to_doc2vec = {
    int(tagged_doc.tags[0]): doc2vec_model.dv[tagged_doc.tags[0]]
    for tagged_doc in tagged_docs
}
```

Listing 10: Εκπαίδευση Doc2Vec και εξαγωγή embeddings

Το Doc2Vec οδήγησε σε μικρή αλλά μετρήσιμη βελτίωση, μειώνοντας τη **μέση αρνητική λογική απώλεια** σε 0.0949 κατά μέσο όρο σε 5-fold cross-validation.

Σημείωση: Σε αυτό το σημείο συνειδητοποιήσαμε ότι είχαμε καλύτερα αποτελέσματα σε cross validation παρά στο kaggle διότι είχαμε data leakage μέσω του γράφου , οπότε στις επόμενες τεχνικές που χρησιμοποιήσαμε τα αποτελέσματα θα είναι χειρότερα σε σχέση με ότι αναφέραμε παραπάνω αλλά στην πραγματικότητα είναι καλύτερα και είναι πιο αντιπροσωπευτικά σε σχέση με το σκορ που μας έβγαζε το kaggle καθώς φροντίσαμε οι ακμές που θα χρησιμοποιούντουσαν ως validation να μην μπουν στον γράφο.

Node2Vec embeddings

Προσπαθήσαμε να υλοποιήσουμε τα Node2Vec[8] embeddings με την παρακάτω υλοποίηση. Έπειτα, για κάθε ζεύγος κόμβων (papers) υπολογίσαμε την ομοιότητα τους με χρήση του cosine similarity. Παρότι η προσέγγιση ήταν λογική, το αποτέλεσμα δεν ήταν ικανοποιητικό καθώς το τελικό σκορ Negative Log-Likelihood (NLL) ήταν 0.1478, υποδεικνύοντας περιθώρια βελτίωσης .

```
from node2vec import Node2Vec
import numpy as np

node2vec = Node2Vec(G, dimensions=64, walk_length=30, num_walks=20,
                    workers=2)
n2v_model = node2vec.fit(window=6, min_count=1, batch_words=4)

paper_to_node2vec = {int(node): n2v_model.wv[str(node)] for node in G.
                     nodes()}

paper_ids = list(paper_to_node2vec.keys())
original_node2vec_matrix = np.array([paper_to_node2vec[pid] for pid in
                                     paper_ids])

np.save("node2vec_original.npy", original_node2vec_matrix)
with open("node2vec_ids.npy", "wb") as f:
    np.save(f, paper_ids)
```

Listing 11: Υλοποίηση Node2Vec embeddings

Για να βελτιώσουμε το score αντι για cosine similarity χρησιμοποιήσαμε concatenate των node2vec embeddings

```
a_node2vec = paper_to_node2vec.get(a, np.zeros(64))
b_node2vec = paper_to_node2vec.get(b, np.zeros(64))

X.append([
    clustering_coeffs.get(a, 0.0),
    clustering_coeffs.get(b, 0.0),
    #closeness Centrality.get(a, 0.0),
    #closeness Centrality.get(b, 0.0),
    compute_adamic_adar(undirected_G, a, b),
    compute_preferential_attachment(undirected_G, a, b),
    cosine_similarity([a_w2v], [b_w2v])[0][0], # Word2Vec sim
    cosine_similarity([a_sci], [b_sci])[0][0], # SciBERT sim
    cosine_similarity([a_doc2vec], [b_doc2vec])[0][0],
    tfidf_sim,
    *a_node2vec,
    *b_node2vec,
    len(authors.get(a, set()) & authors.get(b, set()))
])
y.append(1)
```

Listing 12: Υλοποίηση Node2Vec embeddings με concatenation

Αυτό είχε ως αποτέλεσμα να πέσει το Negative Log-Likelihood (NLL): 0.1242

GNN - GraphSAGE

Το GraphSAGE (Graph Sample and Aggregate)[9] είναι μια μέθοδος που επιτρέπει τη γενίκευση της μάθησης κόμβων σε μη-παρατηρημένους κόμβους χρησιμοποιώντας τεχνικές δειγματοληψίας και συνάθροισης από τους γείτονες.

Τα βασικά χαρακτηριστικά της προσέγγισης GraphSAGE είναι:

- **Specter2 Embeddings**

- Προέρχονται από το προεκπαιδευμένο μοντέλο Specter2.
- Κωδικοποιούν σημασιολογικές πληροφορίες του άρθρου.
- Παρέχουν πλούσια αναπαράσταση του περιεχομένου του κάθε κόμβου.

- **Word2Vec Embeddings**

- Προέρχονται από παραδοσιακό μοντέλο Word2Vec (π.χ. στα keywords ή abstracts).
- Προσφέρουν συμπληρωματική οπτική του περιεχομένου.
- Ενισχύουν τη γενίκευση μέσω διαφορετικής γλωσσικής αναπαράστασης.

- **Δομικά χαρακτηριστικά (Graph Features)**

- **Clustering Coefficient:**

- * Δείχνει πόσο πυκνά συνδέεται ο κόμβος με τους γείτονές του.
 - * Αντανακλά τοπική συνεκτικότητα (π.χ. ερευνητική υποκοινότητα).

- **Closeness Centrality:**

- * Μετρά πόσο «κοντά» είναι ένας κόμβος σε όλους τους άλλους στον γράφο.
 - * Αντανακλά την γενική επιρροή του άρθρου στο δίκτυο παραπομπών.

- **Συνδυασμός όλων των παραπάνω**

- Η ένωση σημασιολογικών και δομικών χαρακτηριστικών δημιουργεί πλήρη αναπαράσταση.
- Παρέχει στο GNN πληροφορία τόσο για το περιεχόμενο όσο και για τη θέση του κόμβου στον γράφο.

- **Οφέλη συνδυασμού χαρακτηριστικών:**

- Καλύτερη αρχικοποίηση των κόμβων.
- Το GNN δεν χρειάζεται να μάθει όλα από το μηδέν.
- Αυξάνεται η ακρίβεια και η δυνατότητα γενίκευσης του μοντέλου.

Δημιουργία edge_index και συναρτήσεων μετατροπής ακμών

```
edge_index = torch.tensor([
    [paper_id_to_index[u], paper_id_to_index[v]]
    for u, v in train_positive_edges
    if u in paper_id_to_index and v in paper_id_to_index
], dtype=torch.long).t().contiguous()

def edge_pairs_to_tensor(edges):
    return torch.tensor([
        [paper_id_to_index[u], paper_id_to_index[v]]
        for u, v in edges
        if u in paper_id_to_index and v in paper_id_to_index
    ], dtype=torch.long).t()

pos_edge_index = edge_pairs_to_tensor(train_positive_edges)
neg_edge_index = edge_pairs_to_tensor(train_negative_edges)
```

Listing 13: Δημιουργία edge_index και συναρτήσεων μετατροπής ακμών

Το παραπάνω τμήμα δημιουργεί:

- Το edge_index, δηλαδή τις ακμές του γραφήματος για το μήνυμα μετάδοσης (message passing).
- Μία βοηθητική συνάρτηση edge_pairs_to_tensor που μετατρέπει λίστες από ακμές (π.χ. θετικές ή αρνητικές) σε tensor μορφή κατάλληλη για PyTorch Geometric.
- Τα pos_edge_index και neg_edge_index χρησιμοποιούνται στην εκπαίδευση του μοντέλου για πρόβλεψη συνδέσεων.

Δημιουργία μεικτών ακμών και ετικετών (labels)

```
all_edge_index = torch.cat([pos_edge_index, neg_edge_index], dim=1)
labels = torch.cat([
    torch.ones(pos_edge_index.shape[1]),
    torch.zeros(neg_edge_index.shape[1])
])
perm = torch.randperm(all_edge_index.shape[1])
all_edge_index = all_edge_index[:, perm]
labels = labels[perm]
```

Listing 14: Συνδυασμός θετικών και αρνητικών ακμών με αντίστοιχες ετικέτες

Αυτό το κομμάτι συνδυάζει τις θετικές και αρνητικές ακμές και δημιουργεί τις αντίστοιχες ετικέτες (1 για θετικές, 0 για αρνητικές). Στη συνέχεια γίνεται ανακάτεμα (shuffle) για τυχαία διάταξη των δειγμάτων κατά την εκπαίδευση.

Ορισμός του μοντέλου GraphSAGE_LinkPredictor

```
class GraphSAGE_LinkPredictor(torch.nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels):
        super().__init__()
        self.conv1 = SAGEConv(in_channels, hidden_channels)
        self.conv2 = SAGEConv(hidden_channels, out_channels)
        self.decoder = torch.nn.Sequential(
            torch.nn.Linear(out_channels * 2, 64),
            torch.nn.ReLU(),
            torch.nn.Linear(64, 1)
        )

    def encode(self, x, edge_index):
        x = F.relu(self.conv1(x, edge_index))
        x = self.conv2(x, edge_index)
        return x

    def decode(self, z, edge_pairs):
        z_i = z[edge_pairs[0]]
        z_j = z[edge_pairs[1]]
        return self.decoder(torch.cat([z_i, z_j], dim=1)).squeeze()
```

Listing 15: Ορισμός του GraphSAGE Link Prediction μοντέλου

Αυτός είναι ο ορισμός του μοντέλου:

- Περιλαμβάνει δύο GraphSAGE convolution layers (conv1, conv2) για την εξαγωγή embeddings.
- Η decode συνάρτηση συνδυάζει τα embeddings δύο κόμβων και τα περνά από ένα MLP για να προβλέψει την ύπαρξη ακμής.

Εκπαίδευση του μοντέλου

```
model = GraphSAGE_LinkPredictor(in_channels=x.shape[1], hidden_channels
                                =256, out_channels=128)
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

model.train()
for epoch in tqdm(range(100), desc="Training GraphSAGE"):
    optimizer.zero_grad()
    z = model.encode(x, edge_index)
    preds = model.decode(z, all_edge_index)
    loss = criterion(preds, labels)
    loss.backward()
    optimizer.step()

    if epoch % 10 == 0:
        with torch.no_grad():
            pred_probs = torch.sigmoid(preds)
            pred_labels = (pred_probs > 0.5).float()
            acc = accuracy_score(labels.cpu(), pred_labels.cpu())
            f1 = f1_score(labels.cpu(), pred_labels.cpu())
            roc_auc = roc_auc_score(labels.cpu(), pred_probs.cpu())
            print(f"Epoch {epoch} | Loss: {loss.item():.4f} | Acc: {acc:.4f} | F1: {f1:.4f} | ROC-AUC: {roc_auc:.4f}")
```

Listing 16: Εκπαίδευση του GraphSAGE μοντέλου

Η διαδικασία εκπαίδευσης του μοντέλου περιλαμβάνει:

- Προώθηση (forward pass) μέσω του encode και decode.
- Υπολογισμό της απώλειας (loss) μέσω criterion (π.χ. BCEWithLogitsLoss).
- Αναδρομική διάδοση (backpropagation) και βελτιστοποίηση.
- Ανά 10 εποχές εμφανίζονται μετρικές αξιολόγησης: ακρίβεια, F1-score και ROC-AUC.

Εξαγωγή τελικών GNN embeddings και συνδυασμός με SPECTER2

```
model.eval()  
with torch.no_grad():  
    gnn_embeddings = model.encode(x, edge_index).cpu()  
  
combined_embeddings = torch.cat([x.cpu(), gnn_embeddings], dim=1)
```

Listing 17: Εξαγωγή GNN embeddings και συνδυασμός με SPECTER2

Μετά την εκπαίδευση:

- Το μοντέλο περνάει σε `evaluation mode`.
- Εξάγονται τα GNN embeddings για κάθε κόμβο.
- Γίνεται συνένωση των GNN embeddings με τα προϋπάρχοντα κειμενικά embeddings (π.χ. από SPECTER2), δημιουργώντας έναν πλουσιότερο συνδυασμό χαρακτηριστικών.

Τα αποτελεσματα που λάβαμε με το παραπάνω GNN ήταν Negative Log-Likelihood (NLL) : 0.1106

Σημείωση: δοκιμάστηκε να συνδυάσουμε GNN μαζί με node2vec αλλά οι βελτιώσεις ήταν σχεδόν μηδαμινές. Εκτός από Graphshage, δοκιμάστηκε και το GCN ωστόσο έβγαζε όσο καλά αποτελέσματα όσο το node2vec.

Χρήσιμα links:

<https://www.analyticsvidhya.com/blog/2024/03/getting-started-with-gnn-implementation/>

<https://www.rohan-paul.com/p/llms-and-graph-neural-networks-for?source=queue>

<https://www.ontotext.com/knowledgehub/fundamentals/what-are-knowledge-graph-embeddings/>

<https://medium.com/data-science/improving-knowledge-graph-embeddings-with-graph-neural-networks-8ffb14804199>

2.3 Feature Combination and Selection

Για την τελική μορφή του πίνακα χαρακτηριστικών X , δημιουργήσαμε διανύσματα χαρακτηριστικών για κάθε ζεύγος κόμβων (a, b) με την εξής δομή:

- Βαθμός κεντρικότητας (*degree centrality*) των κόμβων a και b .
- Συντελεστής συσσωμάτωσης (*clustering coefficient*) των κόμβων a και b .
- Εγγύτητα (*closeness centrality*) των κόμβων a και b .
- Δείκτης Adamic-Adar για το ζεύγος (a, b) .
- cosine similarity σύνδεσης (*preferential attachment*) για το ζεύγος (a, b) .
- cosine similarity μεταξύ των Word2Vec ενσωματώσεων των a και b .
- cosine similarity μεταξύ των SciBERT ενσωματώσεων των a και b .
- cosine similarity μεταξύ των Doc2Vec ενσωματώσεων των a και b .
- TF-IDF ομοιότητα μεταξύ των κειμένων των κόμβων.
- Τα ενσωματωμένα διανύσματα u και v από το GNN (GraphSAGE), για τους κόμβους a και b αντίστοιχα.
- Πλήθος κοινών συγγραφέων μεταξύ των a και b , δηλαδή $|\text{authors}(a) \cap \text{authors}(b)|$.

Τα ενσωματωμένα διανύσματα από το GraphSAGE μοντέλο (emb_u και emb_v) δεν συγκρίθηκαν μέσω cosine similarity, αλλά έγιναν **συνκόλληση** (*concatenation*) για να διατηρηθεί όσο το δυνατόν περισσότερη πληροφορία. Ωστόσο, λόγω περιορισμών μνήμης RAM, εφαρμόστηκε τεχνική **μείωσης διαστατικότητας με PCA** (*Principal Component Analysis*) για να μειωθεί η διάστασή τους σε 50.

```
pca = PCA(n_components=50)
reduced_combined = pca.fit_transform(combined_embeddings)
```

Η τελική λίστα χαρακτηριστικών εισόδου X για κάθε δείγμα είναι ένα διάνυσμα της μορφής:

$$X = [\text{deg}_a, \text{deg}_b, \text{clust}_a, \text{clust}_b, \text{close}_a, \text{close}_b, \text{adamic}(a, b), \\ \text{prefAttach}(a, b), \cos(\text{w2v}_a, \text{w2v}_b), \cos(\text{sci}_a, \text{sci}_b), \cos(\text{doc2vec}_a, \text{doc2vec}_b), \\ \text{tfidfSim}(a, b), (\text{concat}(\text{emb}_u, \text{emb}_v)), |\text{authors}(a) \cap \text{authors}(b)|]$$

Το αντίστοιχο label $y = 0$ ή 1 , ανάλογα με το αν υπάρχει ή όχι citation.

3 Modeling and Comparison

Μέχρι στιγμής το Negative Log-Likelihood (NLL) με GNN : 0.1106 το έχουμε καταφέρει με ένα απλό LogisticRegression(max_iter=1500, random_state=42)

Logistic Regression και Ρύθμιση Υπερπαραμέτρων

Η **λογιστική παλινδρόμηση (Logistic Regression)** [10] είναι ένα απλό αλλά αποτελεσματικό γραμμικό μοντέλο για δυαδική ταξινόμηση. Το μοντέλο προβλέπει την πιθανότητα ότι ένα δείγμα ανήκει σε μία από δύο κλάσεις, εφαρμόζοντας τη σιγμοειδή συνάρτηση (sigmoid) στην έξοδο μιας γραμμικής συνάρτησης:

$$P(y = 1 | \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$

Το μοντέλο εκπαιδεύεται ώστε να ελαχιστοποιήσει τη λογιστική απώλεια (log-loss), γνωστή και ως **αρνητική λογική πιθανοφάνεια (Negative Log-Likelihood)**.

Κατά την εκπαίδευση εφαρμόζεται κανονικοποίηση (regularization) για την αποφυγή υπερπροσαρμογής, με την προσθήκη ενός όρου στην απώλεια. Ο βαθμός της κανονικοποίησης ελέγχεται από την υπερπαραμέτρο **C**.

Η υπερπαραμέτρος C

Η παράμετρος C ρυθμίζει την ισχύ της κανονικοποίησης:

$$\text{Loss}_{\text{reg}} = \text{LogLoss} + \frac{1}{2C} \|\mathbf{w}\|_2^2$$

Όπου: - LogLoss είναι η λογιστική απώλεια (χωρίς κανονικοποίηση) - $\|\mathbf{w}\|_2^2$ είναι ο L_2 -κανονικοποιητικός όρος - C είναι η αντιστροφή της κανονικοποίησης: μικρότερες τιμές της C σημαίνουν **ισχυρότερη κανονικοποίηση**, ενώ μεγαλύτερες τιμές οδηγούν σε **χαλαρότερη κανονικοποίηση**.

Πείραμα με C = 0.02

Χρησιμοποιώντας το μοντέλο:

```
LogisticRegression(C=0.02, max_iter=1500, random_state=42)
```

επιτύχαμε την καλύτερη απόδοση στο validation set, με τιμή:

Negative Log-Likelihood (NLL) = **0.1088**

Αυτό δείχνει ότι η συγκεκριμένη τιμή της υπερπαραμέτρου C οδήγησε σε καλή ισορροπία μεταξύ κανονικοποίησης και προσαρμογής στο δεδομένο σύνολο χαρακτηριστικών.

Random Forest Classifier και Βελτιστοποίηση Υπερπαραμέτρων

Ο **Random Forest Classifier**[2] είναι ένα σύνολο (ensemble) από πολλά *decision trees*, τα οποία εκπαιδεύονται ανεξάρτητα και στη συνέχεια συνδυάζουν τις προβλέψεις τους μέσω πλειοψηφίας (για classification). Η χρήση πολλών δέντρων αυξάνει τη γενικευσιμότητα και μειώνει τον κίνδυνο υπερπροσαρμογής.

Ο Random Forest είναι γνωστός για την ικανότητά του να χειρίζεται δεδομένα με πολλαπλά χαρακτηριστικά, χωρίς σημαντική ανάγκη για κανονικοποίηση ή scaling.

Το μοντέλο που έδωσε την καλύτερη απόδοση με βάση το **Negative Log-Likelihood (NLL)** είναι το εξής:

```
RandomForestClassifier(  
    n_estimators=21,  
    max_depth=19,  
    min_samples_split=5,  
    min_samples_leaf=20,  
    random_state=42)
```

Επεξήγηση Υπερπαραμέτρων

- `n_estimators=21`: Αριθμός δέντρων στο δάσος. Περισσότερα δέντρα μειώνουν τη διακύμανση αλλά αυξάνουν τον χρόνο εκπαίδευσης.
- `max_depth=19`: Μέγιστο βάθος κάθε δέντρου. Περιορίζει την πολυπλοκότητα του μοντέλου και προλαμβάνει την υπερπροσαρμογή.
- `min_samples_split=5`: Ελάχιστος αριθμός δειγμάτων που απαιτούνται για να γίνει split σε έναν εσωτερικό κόμβο.
- `min_samples_leaf=20`: Ελάχιστος αριθμός δειγμάτων που πρέπει να υπάρχουν σε ένα φύλλο (τελικό κόμβο). Αυξάνει την γενίκευση.
- `random_state=42`: Σταθερότητα αποτελεσμάτων μέσω καθορισμένου seed.

Αποτέλεσμα

Με τις παραπάνω υπερπαραμέτρους, ο Random Forest πέτυχε:

$$\text{Negative Log-Likelihood (NLL)} = \mathbf{0.1152}$$

Παρότι η απόδοση ήταν καλή, ήταν ελαφρώς χειρότερη από την καλύτερη τιμή που επιτεύχθηκε με λογιστική παλινδρόμηση (NLL = 0.1088). Αυτό δείχνει ότι το απλό γραμμικό μοντέλο (Logistic Regression) με κατάλληλα χαρακτηριστικά και κανονικοποίηση μπορεί να ξεπεράσει πιο σύνθετα μοντέλα όπως το Random Forest.

XGBoost Classifier και Βελτιστοποίηση Υπερπαραμέτρων

Ο **XGBoost (Extreme Gradient Boosting)**[3] είναι ένας αλγόριθμος μηχανικής μάθησης βασισμένος στο *gradient boosting*, που εκπαιδεύει ακολουθιακά πολλά ασθενή μοντέλα (συνήθως δέντρα απόφασης), ώστε κάθε νέο μοντέλο να διορθώνει τα σφάλματα του προηγούμενου. Οι υπερπαραμέτροι που οδήγησαν στο καλύτερο αποτέλεσμα ήταν:

```
XGBClassifier(  
    n_estimators=180,  
    max_depth=3,  
    learning_rate=0.06,  
    subsample=0.8,  
    colsample_bytree=0.9,  
    eval_metric='logloss',  
    random_state=42)
```

Επεξήγηση Υπερπαραμέτρων

- `n_estimators=180`: Αριθμός δέντρων στο boosting σύνολο. Περισσότερα δέντρα σημαίνουν πιο σύνθετο μοντέλο.
- `max_depth=3`: Μέγιστο βάθος κάθε δέντρου. Μικρό βάθος περιορίζει την υπερπροσαρμογή.
- `learning_rate=0.06`: Ρυθμός εκμάθησης (η), ελέγχει πόσο επηρεάζει κάθε νέο δέντρο το συνολικό μοντέλο. Μικρές τιμές οδηγούν σε πιο σταδιακή εκμάθηση.
- `subsample=0.8`: Ποσοστό δειγμάτων που χρησιμοποιούνται για την εκπαίδευση κάθε δέντρου. Βοηθά στην αποφυγή overfitting.
- `colsample_bytree=0.9`: Ποσοστό χαρακτηριστικών που χρησιμοποιούνται σε κάθε δέντρο. Εισάγει επιπλέον τυχαιότητα.

Αποτέλεσμα

Με τις παραπάνω παραμέτρους, το XGBoost πέτυχε:

$$\text{Negative Log-Likelihood (NLL)} = \mathbf{0.1020}$$

Το σκορ αυτό είναι το **καλύτερο** που επιτεύχθηκε στο project μέχρι στιγμής και στο kaggle, ξεπερνώντας ακόμα και τη Logistic Regression (NLL = 0.1088) και τον Random Forest (NLL = 0.1152). Η ικανότητα του XGBoost να μαθαίνει πολύπλοκα patterns και να αποφεύγει την υπερπροσαρμογή μέσω ρυθμιζόμενης τυχαιότητας και μικρού learning rate, συνέβαλε στη βελτιωμένη απόδοση.

LightGBM Classifier και Βελτιστοποίηση Υπερπαραμέτρων

Το **LightGBM (Light Gradient Boosting Machine)**[11] είναι ένας αλγόριθμος boosting που σχεδιάστηκε για να παρέχει υψηλή ταχύτητα και χαμηλή χρήση μνήμης, ακόμα και σε datasets με μεγάλο αριθμό παρατηρήσεων και χαρακτηριστικών κάτι που βοηθάει στην δικιά μας περίπτωση όπου έχουμε μεγάλου μεγέθους δεδομένα κυρίως λόγω των gnn embeddings. Σε αντίθεση με το παραδοσιακό Gradient Boosting, χρησιμοποιεί τεχνικές όπως *leaf-wise tree growth* αντί για *level-wise*, προσφέροντας καλύτερη απόδοση και ακρίβεια.

Ο βελτιστοποιημένος classifier που χρησιμοποιήθηκε ήταν:

```
LGBMClassifier(  
    num_leaves=27,  
    n_estimators=120,  
    learning_rate=0.05,  
    colsample_bytree=1,  
    random_state=42,  
    verbose=-1)
```

Επεξήγηση Υπερπαραμέτρων

- `num_leaves=27`: Ο μέγιστος αριθμός φύλλων ανά δέντρο. Μεγαλύτερος αριθμός φύλλων αυξάνει την πολυπλοκότητα του μοντέλου.
- `n_estimators=120`: Ο αριθμός των δέντρων (boosting rounds) που εκπαιδεύονται.
- `learning_rate=0.05`: Ρυθμός εκμάθησης που ελέγχει πόσο επηρεάζει κάθε νέο δέντρο το σύνολο. Μικρές τιμές βοηθούν στη σταθερότερη εκμάθηση.
- `colsample_bytree=1`: Ποσοστό χαρακτηριστικών που χρησιμοποιούνται ανά δέντρο. Τιμή 1 σημαίνει χρήση όλων των χαρακτηριστικών.

Αποτέλεσμα

Με τις παραπάνω παραμέτρους, ο LightGBM classifier πέτυχε:

$$\text{Negative Log-Likelihood (NLL)} = \mathbf{0.1016}$$

Πρόκειται για το **καλύτερο αποτέλεσμα στο τοπικό validation set** σε σύγκριση με όλους τους άλλους classifiers, ακόμα και από το XGBoost. Ωστόσο, κατά την υποβολή στο Kaggle, ο LightGBM εμφάνισε **χειρότερη γενίκευση** σε σχέση με το XGBoost, γεγονός που πιθανόν οφείλεται σε μικρή υπερπροσαρμογή στα δεδομένα εκπαίδευσης ή διαφορές στη διανομή των test δεδομένων.

CatBoost Classifier και Βελτιστοποίηση Υπερπαραμέτρων

Το **CatBoost (Categorical Boosting)** [17] είναι και αυτός ένας gradient boosting αλγόριθμος και είναι κυρίως αποδοτικός σε σύνολα δεδομένων που περιέχουν κατηγορικά χαρακτηριστικά. Βασίζεται σε decision trees όπως οι XGBoost και LightGBM, αλλά χρησιμοποιεί καινοτόμες τεχνικές όπως *ordered boosting* για να περιορίσει την υπερπροσαρμογή και να βελτιώσει τη γενίκευση.

Ο βελτιστοποιημένος CatBoost classifier που χρησιμοποιήθηκε ήταν:

```
CatBoostClassifier(  
    iterations=150,  
    depth=3,  
    learning_rate=0.07,  
    subsample=0.8,  
    colsample_bylevel=0.9,  
    eval_metric='Logloss',
```

Επεξήγηση Υπερπαραμέτρων

- `iterations=150`: Ο συνολικός αριθμός boosting iterations (ή δέντρων).
- `depth=3`: Το μέγιστο βάθος κάθε δέντρου. Μικρό βάθος σημαίνει απλούστερα δέντρα και μεγαλύτερη γενίκευση.
- `learning_rate=0.07`: Ο ρυθμός εκμάθησης, καθορίζει πόσο επηρεάζει κάθε νέο δέντρο το σύνολο.
- `subsample=0.8`: Ποσοστό παρατηρήσεων που δειγματοληπτούνται σε κάθε iteration (προστασία από overfitting).
- `colsample_bylevel=0.9`: Ποσοστό χαρακτηριστικών που χρησιμοποιούνται σε κάθε επίπεδο των δέντρων.

Αποτέλεσμα

Με τις παραπάνω παραμέτρους, ο CatBoost πέτυχε:

$$\text{Negative Log-Likelihood (NLL)} = \mathbf{0.1013}$$

Το παραπάνω σκορ αποτελεί **το καλύτερο αποτέλεσμα στο τοπικό validation set** ανάμεσα σε όλους τους classifiers. Ωστόσο, στην υποβολή στο Kaggle, ο XGBoost εξακολουθεί να παρουσιάζει καλύτερη γενίκευση, υποδηλώνοντας ότι τα χαρακτηριστικά του dataset ευνοούν την πιο κλασική προσέγγιση του XGBoost στο unseen test set.

HistGradientBoosting Classifier και Βελτιστοποίηση Υπερπαραμέτρων

Ο **HistGradientBoostingClassifier**[7] είναι μια υλοποίηση του Gradient Boosted Decision Trees (GBDT) της βιβλιοθήκης `scikit-learn`, η οποία χρησιμοποιεί binning (ιστογράμματα) για να επιταχύνει την εκπαίδευση, καθιστώντας τον ιδιαίτερα αποδοτικό για μεγάλα datasets. Είναι εμπνευσμένος από υλοποιήσεις όπως το LightGBM και XGBoost.

Ο αρχικός βελτιστοποιημένος ταξινομητής είχε τις παρακάτω υπερπαραμέτρους:

```
HistGradientBoostingClassifier(  
    learning_rate=0.06,  
    max_iter=140,  
    max_leaf_nodes=11,  
    max_depth=None,  
    min_samples_leaf=4,  
    l2_regularization=0.0,  
    early_stopping=True,  
    random_state=42,  
    verbose=1)
```

Επεξήγηση Υπερπαραμέτρων

- **learning_rate**: Καθορίζει πόσο συμβάλλει κάθε δέντρο στην τελική πρόβλεψη. Μικρότερες τιμές βελτιώνουν τη γενίκευση αλλά απαιτούν περισσότερα δέντρα.
- **max_iter**: Συνολικός αριθμός boosting iterations (ή δέντρων).
- **max_leaf_nodes**: Ο μέγιστος αριθμός φύλλων ανά δέντρο. Περιορίζει την πολυπλοκότητα κάθε δέντρου.
- **max_depth**: Το μέγιστο βάθος κάθε δέντρου (None σημαίνει απεριόριστο βάθος, περιορίζεται μόνο από τα φύλλα).
- **min_samples_leaf**: Ελάχιστος αριθμός δειγμάτων που απαιτούνται για να σχηματιστεί ένα φύλλο.
- **l2_regularization**: Ελέγχει την κανονικοποίηση για την αποφυγή overfitting.
- **early_stopping**: Τερματίζει πρόωρα την εκπαίδευση αν δεν βελτιώνεται η απόδοση στο validation set.

Αποτέλεσμα

Ο παραπάνω ταξινομητής πέτυχε:

$$\text{Negative Log-Likelihood (NLL)} = \mathbf{0.1003}$$

το οποίο αποτελεί **το καλύτερο σκορ στο τοπικό validation set και στο Kaggle** μέχρι στιγμής, ξεπερνώντας τόσο τον XGBoost όσο και τον CatBoost.

Για ακόμη καλύτερη απόδοση στο Kaggle, δοκιμάστηκε μια ελαφρώς τροποποιημένη εκδοχή του μοντέλου:

```
HistGradientBoostingClassifier(  
    learning_rate=0.07,  
    max_iter=220,  
    max_leaf_nodes=12,  
    max_depth=None,  
    min_samples_leaf=4,  
    l2_regularization=0.0,  
    early_stopping=True,  
    random_state=42,  
    verbose=1)
```

Η συγκεκριμένη διαμόρφωση αποτέλεσε τη **βέλτιστη λύση ως προς την γενίκευση στο τελικό Kaggle test set**, αποδίδοντας καλύτερα από όλους τους υπόλοιπους classifiers που δοκιμάστηκαν στο πλαίσιο του έργου.

Πειραματική Δοκιμή MLP για Κατηγοριοποίηση

Σε δεύτερο στάδιο, δοκιμάστηκε εντελώς πειραματικά η χρήση των MLP - Multi-Layer Perceptrons [18] για το έργο της κατηγοριοποίησης. Ο στόχος ήταν να εξεταστεί η ικανότητα των MLP να ανταγωνιστούν πιο παραδοσιακούς ή/και πιο εξειδικευμένους ταξινομητές.

Για να εκτιμήσουμε αρχικά το καλύτερο tuning , υλοποιήθηκε αναζήτηση σε πλέγμα (Grid Search) πάνω σε διάφορες παραμέτρους :

- **hidden_layer_sizes**: (64), (128), (256), (128, 64), (256, 128), (256, 128, 64)
- **activation**: relu, tanh
- **solver**: adam, sgd
- **alpha** (L2 penalty): 0.0001, 0.001
- **learning_rate_init**: 0.001, 0.01

Το μοντέλο εκπαιδευόταν με `early_stopping=True` και `max_iter=100`.

Παρατηρήθηκε ότι το MLP είχε γενικά καλή συμπεριφορά όταν χρησιμοποιούσε:

- **Μικρό βάθος** (1–2 κρυφά επίπεδα)
- **SGD optimizer** αντί του adam
- **Learning rate = 0.001**

Ορισμένες από τις καλύτερες διαμορφώσεις (με βάση το NLL) ήταν οι εξής:

- `hidden_layer_sizes=(128, 64)`, `activation=relu`, `solver=sgd`, `alpha=0.0001`, `learning_rate=0.001` με **NLL = 0.1672**
- `hidden_layer_sizes=(128,)`, `activation=relu`, `solver=sgd`, `alpha=0.0001`, `learning_rate=0.001` με **NLL = 0.1681**
- `hidden_layer_sizes=(256, 128, 64)`, `activation=relu`, `solver=sgd`, `alpha=0.0001`, `learning_rate=0.001` με **NLL = 0.1841**

Αντίθετα, η χρήση του adam οδήγησε γενικά σε υψηλότερες τιμές NLL, ειδικά όταν το δίκτυο γινόταν βαθύτερο, ενώ και τα μεγάλα learning rates (0.01) ήταν πιο ασταθή.

Βέλτιστη Διαμόρφωση MLP

Μετά από εκτεταμένο πειραματισμό με διαφορετικές υπερπαραμέτρους του πολυεπίπεδου perceptron (MLP), η καλύτερη απόδοση επιτεύχθηκε με τη χρήση ενός πολύ μικρού αριθμού νευρώνων και μιας μόνο εποχής, γεγονός που υποδεικνύει ότι το δίκτυο υπερεκπαιδεύεται εύκολα λόγω του μεγάλου πλήθους δεδομένων.

- **Απώλεια Log Loss:** 0.1031
- **Βέλτιστες υπερπαραμέτροι:**
 - `hidden_layer_sizes:` (3, 4)
 - `activation:` ReLU
 - `solver:` Adam
 - `alpha:` 0.001
 - `learning_rate_init:` 0.001
 - `momentum:` 0.6 (αν και αφορά κυρίως τον SGD, δεν είχε επίδραση στον Adam)

Η συγκεκριμένη διαμόρφωση δείχνει ότι το MLP ευνοείται από μια απλή αρχιτεκτονική όταν το πλήθος των δειγμάτων είναι μεγάλο. Μεγαλύτερα δίκτυα οδηγούν σε φαινόμενα υπερεκπαίδευσης (overfitting), χωρίς να βελτιώνουν την ακρίβεια .

Η αλήθεια είναι ότι το MLP δεν δοκιμάστηκε στο kaggle καθώς δεν ξεπέρασε το HistGradientBoosting και η βελτιστοποίηση παραμέτρων που θα μας δώσει τα καλύτερα αποτελέσματα στο validation test θα διαφέρει πάρα πολύ από την βελτιστοποίηση παραμέτρων που θα μας δώσει τα καλύτερα αποτελέσματα στο kaggle . Γενικά στα MLP είναι δύσκολο το tuning , παρά το γεγονός ότι το MLP δεν αποτελεί state-of-the-art προσέγγιση για embeddings, απέδωσε αξιοπρεπώς, γεγονός που ενισχύει τη διαπίστωση ότι ακόμα και απλά νευρωνικά δίκτυα μπορούν να είναι αποδοτικά σε κατάλληλα διαμορφωμένα χαρακτηριστικά.

Ανεπιτυχής Δοκιμή με SVM

Δοκιμάστηκε η χρήση του Support Vector Machine (SVM)[5] ως ταξινομητή για το συγκεκριμένο πρόβλημα, με στόχο τη σύγκριση της απόδοσής του με τα νευρωνικά δίκτυα. Ωστόσο, η εφαρμογή του αποδείχθηκε πρακτικά ανέφικτη λόγω του υπερβολικού χρόνου εκπαίδευσης.

Πιο συγκεκριμένα, ένα απλό πείραμα με βασικές παραμέτρους (χωρίς βελτιστοποίηση) ξεκίνησε να τρέχει αλλά ακόμα και μετά από **7–8 ώρες** δεν είχε ολοκληρώσει. Η συμπεριφορά αυτή αποδίδεται στο μεγάλο πλήθος δειγμάτων στο dataset, το οποίο καθιστά την εκπαίδευση των SVM εξαιρετικά χρονοβόρα.

Τα Support Vector Machines είναι πανίσχυροι αλγόριθμοι ταξινόμησης, ιδιαίτερα αποτελεσματικοί σε προβλήματα μικρού ή μεσαίου μεγέθους και με σαφώς διαχωρίσιμες κατηγορίες. Η βασική τους αρχή βασίζεται στην εύρεση του υπερεπιπέδου που μεγιστοποιεί το περιθώριο μεταξύ των κλάσεων.

Ωστόσο, η υπολογιστική πολυπλοκότητα της εκπαίδευσης ενός SVM αυξάνεται τουλάχιστον **γραμμικά έως και τετραγωνικά** ως προς το πλήθος των δειγμάτων, ανάλογα με την υλοποίηση (π.χ. kernel-based μέθοδοι). Επιπλέον, η χρήση kernels (π.χ. RBF) σε μεγάλα σύνολα δεδομένων επιβαρύνει σημαντικά τη μνήμη και τον χρόνο επεξεργασίας, καθώς απαιτεί τον υπολογισμό όλων των ζευγαριών δειγμάτων.

Για τους παραπάνω λόγους, η επιλογή του SVM εγκαταλείφθηκε, καθώς ήταν πρακτικά απαγορευτική για το μέγεθος του dataset.

References

- [1] Iz Beltagy, Kyle Lo, and Arman Cohan. Scibert: A pretrained language model for scientific text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
- [2] Leo Breiman. Random forests. *Machine Learning*, 2001.
- [3] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [4] Arman Cohan, Sebastian Feldman, Kyle Lo, David Belanger, Andrew Yates, Gargi Ghosh, Daniel Cushing, and Waleed Ammar. Specter 2: Embedding-based document retrieval for scholarly literature. *arXiv preprint arXiv:2304.12289*, 2023.
- [5] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 1995.
- [6] Linton C Freeman. Centrality in social networks: Conceptual clarification. *Social Networks*, 1979.
- [7] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 2001.

- [8] Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016.
- [9] William L Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, 2017.
- [10] David W. Hosmer, Stanley Lemeshow, and Rodney X. Sturdivant. *Applied Logistic Regression*. Wiley, 2013.
- [11] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, 2017.
- [12] Quoc Le and Tomas Mikolov. Distributed representations of sentences and documents. In *International Conference on Machine Learning*, 2014.
- [13] David Liben-Nowell and Jon Kleinberg. The link-prediction problem for social networks. *Journal of the American Society for Information Science and Technology*, 2007.
- [14] Christopher D Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [15] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *Proceedings of Workshop at ICLR*, 2013.
- [16] Karl Pearson. Liii. on lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 1901.
- [17] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. Catboost: unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, 2018.
- [18] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 1986.
- [19] Duncan J Watts and Steven H Strogatz. Collective dynamics of ‘small-world’ networks. *Nature*, 1998.